As you learned in chapter 2, modifiers are a special programming language structure offered by Solidity for implementing verification and validation rules in a smart contract. Let's first review how to define them and then dig into how to use them effectively.

Listing 3.3 shows the syntax of a modifier, which is somewhat like a function definition:

- It has a header line with a name and parameter list.
- It has a body that specifies the conditions to be checked within a `require` statement.
- This line is followed by `___;` , which represents the code that follows the modifier at the actual location where it would be used. This symbol represents the code that the modifier guards.

The next listing also shows an example use of a modifier. Here, it is verifying that the state of the ballot process is in the correct phase, as specified by the parameter `reqPhase`.

Listing 3.3 Modifier definition syntax and example

```
modifier name_of_modifier (parameters)    ❶
{ require { conditions_to_be_checked};
    _;
}

modifier validPhase(Phase reqPhase)       ❷
{ require(state == reqPhase);
  _;
}
```

❶ Modifier syntax

❷ Actual modifier definition for validPhase rule

Why separate the modifier definition from the function definition? The idea is to separate the verification, validation,